

Cómo piensa el modelo · SOUL.md con criterio · AGENTS.md como código · Context window ·
Diseño de conversación · Testing

4

GUÍA 2 — MÓDULO 4

Prompt Engineering y Diseño de Conversación

Instrucciones que el modelo realmente sigue — y por qué las otras no

I — Cómo construye OpenClaw el contexto del modelo

Antes de escribir una sola instrucción, hay que entender cómo llega esa instrucción al modelo. OpenClaw construye un sistema de prompt diferente en cada turno, ensamblando varias piezas en un orden específico. Lo que no está en ese ensamblado, el modelo no lo ve.

Componente	Qué contiene	Tokens aprox.	Se puede modificar
Sistema base (OpenClaw)	Lista de herramientas, reglas de seguridad, información de runtime, hora actual	~9.600 tokens	No directamente
AGENTS.md	Rutinas, reglas de comportamiento, protocolos de la sesión	~436 tokens (ejemplo real)	Sí — es tuyo
SOUL.md	Personalidad, tono, valores, identidad del agente	~228 tokens (ejemplo real)	Sí — es tuyo
TOOLS.md	Notas sobre herramientas locales — puede truncarse si es grande	Hasta 20.000 chars (límite por archivo)	Sí
IDENTITY.md / USER.md / MEMORY.md	Identidad del agente, datos del usuario, memoria curada	Variable — crecen con el tiempo	Sí
Lista de skills	Solo los nombres y descripciones — el contenido completo se carga bajo demanda	~195 chars base + ~97 por skill	Sí — vía SKILL.md
Historial de conversación	Todos los mensajes + llamadas a herramientas de la sesión activa	Variable — puede crecer sin límite	Via historyLimit

→ La lección clave de la documentación oficial

Todo lo que está inyectado en el contexto consume tokens en cada turno — no solo cuando se menciona. Un agente con MEMORY.md extenso, TOOLS.md largo, 30 skills y muchas herramientas puede agotar el presupuesto de contexto antes de que empiece la tarea real. Un usuario reportó gastar 346K tokens en una tarea que debería haber costado 28K — por tener historyLimit en 2 (el agente buscaba los mismos archivos repetidamente).

Usar /context list para ver exactamente qué está consumiendo el contexto y cuánto pesa cada componente.

SOUL.md — personalidad que el modelo realmente sigue

II — SOUL.md con criterio

SOUL.md es el archivo que define quién es el agente. Se inyecta en cada turno. Un SOUL.md genérico produce un agente genérico. Un SOUL.md bien escrito produce un agente que suena como alguien — con criterio propio, estilo consistente y límites claros.

La diferencia entre decorativo y funcional

SOUL.md decorativo — el modelo lo ignora	SOUL.md funcional — el modelo lo sigue
Sos un asistente amable y profesional.	Cuando alguien pide algo, ejecutá primero y explicá después. No preguntés si ya tenés suficiente información para actuar.
Tenés muchos años de experiencia en negocios.	Si alguien propone una idea que tiene un problema obvio, señalalo directamente antes de ayudar a ejecutarla.
Siempre respondés con honestidad y claridad.	Las respuestas largas van en el canal de WhatsApp. Si el mensaje tiene más de 5 líneas, envíalo como documento o dividilo en partes numeradas.
Sos empático y entendes las necesidades del usuario.	No uses emojis en respuestas técnicas o de trabajo. Usá emojis solo en conversaciones casuales.

La diferencia no es el tono de lo que escribís — es si la instrucción cambia algo concreto en el comportamiento. "Amable" no le dice al modelo qué hacer diferente. "Ejecutá primero, explicá después" sí.

Estructura de un SOUL.md real

```
# SOUL.md

## Identidad
$ Me llamo Kero. Soy el asistente de [nombre de la empresa].
$ No soy un chatbot genérico — tengo criterio y opiniones.

## Cómo actúo
$ - Ejecuto primero, explico después. Si tengo suficiente información, actúo.
$ - Si veo un problema evidente en lo que me piden, lo digo antes de ejecutar.
$ - Mis respuestas son directas. Sin relleno, sin "¡Claro que sí!" al principio.
$ - En conversaciones técnicas: sin emojis. En conversaciones casuales: usarlos está bien.
```

```
## Formato de respuestas
```

- \$ - Respuestas cortas (menos de 4 líneas): texto directo en el chat.
- \$ - Respuestas largas o estructuradas: dividir las en secciones numeradas.
- \$ - Código o comandos: siempre en bloque separado, nunca inline en el texto.

```
## Qué no hago
```

- \$ - No doy respuestas vagas cuando hay una respuesta concreta posible.
- \$ - No pregunto "¿en qué puedo ayudarte?" — el usuario ya dijo qué necesita.
- \$ - No repito lo que el usuario acaba de decir antes de responder.

→ SOUL.md como código — versionado y testeable

Un ingeniero que trabajó meses con OpenClaw lo describió así: 'Antes pensaba que el camino a mejores agentes era mejores modelos. Después de estudiar OpenClaw, estoy convencido de que el camino es mejor context engineering. El mismo modelo con prompts más inteligentes supera consistentemente a un modelo más grande con prompts débiles.'

Tratá SOUL.md como código: versionalo en git, documentá por qué cada instrucción está ahí, y cuando algo no funciona como esperás, editá el archivo — no la conversación.

AGENTS.md — rutinas que el modelo ejecuta

III — AGENTS.md como código de comportamiento

AGENTS.md define qué hace el agente y cuándo. Si SOUL.md es la personalidad, AGENTS.md es el manual de operación. La diferencia con la Guía 1 es el nivel de precisión: en nivel avanzado, cada instrucción en AGENTS.md debe ser tan específica que no deje ambigüedad sobre qué hacer.

Instrucciones ambiguas vs. instrucciones ejecutables

Ambigua — el modelo improvisa	Ejecutable — el modelo sigue
Respondé rápido a los mensajes urgentes.	Si un mensaje contiene las palabras 'urgente', 'crítico' o 'cayó el servidor': responder en menos de 2 minutos, notificar al usuario por WhatsApp aunque sea fuera de horario, registrar en MEMORY.md con la etiqueta [URGENTE].
Mantené un registro de las tareas.	Al finalizar cada tarea: agregar una línea a MEMORY.md con formato: [FECHA] [TAREA] [RESULTADO]. Máximo una línea por tarea.
Coordiná con el calendario.	Antes de sugerir una reunión: verificar disponibilidad en Google Calendar en los próximos 5 días hábiles. Si no hay slots libres de 45 minutos, notificar al usuario antes de responder.
Manejá los errores con cuidado.	Si una herramienta devuelve un error: registrar el error exacto, reintentar una vez, si persiste notificar al usuario con el código de error y no reintentar más.

Secciones recomendadas en AGENTS.md avanzado

```
# AGENTS.md

## Inicio de sesión
# Qué hace el agente al arrancar cada sesión
$ Al iniciar: leer SECURITY.md y PRIVACY.md.
$ Si es la primera sesión del día: revisar pendientes en MEMORY.md.

## Clasificación de mensajes
# Cómo procesa el agente lo que llega
$ Si el mensaje es una pregunta factual: responder directamente, sin herramientas.
$ Si requiere búsqueda o acción: confirmar el objetivo antes de actuar.
$ Si es ambiguo: pedir clarificación con una sola pregunta específica.

## Protocolos por tipo de tarea
```

```
# Instrucciones específicas según el tipo de trabajo
$ Tarea de investigación: buscar al menos 3 fuentes antes de responder.
$ Tarea de escritura: preguntar el tono y la extensión esperada antes de empezar.
$ Tarea técnica: confirmar el entorno (SO, versiones) si no está en MEMORY.md.

## Límites de autonomía
# Cuándo actuar solo y cuándo pedir confirmación
$ Actuar sin preguntar: lectura de archivos, búsqueda web, respuestas informativas.
$ Preguntar antes: enviar emails, modificar archivos, ejecutar comandos de shell.
$ Siempre preguntar: borrar archivos, enviar mensajes a terceros, compras.

## Cierre de sesión
$ Al finalizar: si se generaron tareas pendientes, registrarlas en MEMORY.md.
```

Context window — el recurso más escaso

IV — Context window: qué es y cómo gestionarlo

La context window es el límite de información que el modelo puede procesar en un turno. Todo lo que está dentro influye en la respuesta. Todo lo que está fuera, el modelo no lo ve. En OpenClaw, esto tiene consecuencias prácticas directas: un agente mal configurado puede consumir su context window en los archivos de workspace antes de empezar la tarea.

Cómo diagnosticar el uso del contexto

```
# Ver qué está consumiendo el contexto ahora:
$ /context list

# Ver el desglose detallado por archivo y herramienta:
$ /context detail

# Ver qué tan lleno está el contexto en este momento:
$ /estado

# Ver el consumo de tokens por respuesta:
$ /uso tokens

# Ejemplo de lo que muestra /context list:
# AGENTS.md: 436 tokens (OK)
# SOUL.md: 228 tokens (OK)
# TOOLS.md: TRUNCADO | 13.553 tokens raw | 5.241 inyectados
# MEMORY.md: 2.847 tokens (atención — está creciendo)
# Historial de sesión: 34.200 tokens (sesión larga)
```

Las cinco causas más comunes de context overflow

Causa	Cómo detectarla	Cómo resolverla
MEMORY.md que creció sin control	Pesa más de 2.000 tokens en /context list	Revisar y resumir entradas viejas. Mover información permanente a archivos específicos en lugar de acumular en MEMORY.md.
TOOLS.md demasiado largo	Aparece como TRUNCADO en /context list	Dividir en archivos temáticos. Solo incluir lo que el agente usa activamente.
historyLimit muy bajo	El agente busca repetidamente los mismos archivos	Aumentar historyLimit en la configuración. Un valor de 2 hace que el agente reempiece constantemente.

Sesiones que no se cierran	La sesión lleva muchos intercambios con llamadas a herramientas	Usar /nuevo para sesiones muy largas. La información importante se preserva en MEMORY.md antes de cerrar.
Demasiados skills instalados	La lista de skills pesa muchos tokens en /context detail	Cada skill agrega ~97 chars a cada turno. Desinstalar skills que no se usan. Mantener solo los relevantes para ese agente.

Compactación — lo que pasa cuando el contexto se llena

Cuando la context window se llena, OpenClaw ejecuta compactación: resume los turnos más viejos de la conversación en entradas comprimidas para liberar espacio. Esto preserva el contenido semántico pero puede perder detalles específicos, incluyendo — según un issue abierto en el repositorio oficial — instrucciones críticas de seguridad o restricciones activas que estaban en el historial.

⚠ Compactación y pérdida de restricciones — caso documentado

Un usuario que corre un agente persistente desde enero 2026 documentó en el repositorio oficial que la compactación puede eliminar silenciosamente instrucciones críticas del historial: reglas de seguridad, restricciones activas, directivas 'no actuar'. El agente continúa pero sin esas restricciones en contexto.

Su solución interim: un archivo MORNING-TAPE.md que se lee al inicio de cada sesión con las restricciones críticas — y una instrucción en AGENTS.md de releerlo antes de cualquier acción irreversible. Reduce el riesgo ~40-50%. La solución definitiva (slots de contexto persistentes) está propuesta pero no implementada aún.

Diseño de conversación — flujos complejos

V — Diseño de flujos conversacionales

Un flujo conversacional es la secuencia de pasos que sigue el agente para resolver un tipo de pedido. Sin diseño explícito, el agente improvisa — a veces bien, a veces no. En nivel avanzado, los flujos más importantes se documentan en AGENTS.md como protocolos reproducibles.

Anatomía de un flujo bien diseñado

Elemento del flujo	Para qué sirve
Trigger	La condición que activa el flujo. Específica, no ambigua. Ej: 'cuando el mensaje contiene una fecha + tarea + persona'.
Verificación previa	Qué debe chequear el agente antes de actuar. Ej: 'confirmar que el usuario tiene permisos para esta acción'.
Pasos con orden explícito	La secuencia exacta. Numerada. Con condiciones de ramificación cuando el flujo puede bifurcarse.
Condiciones de salida	Cuándo termina el flujo — éxito, error, o escalamiento al usuario.
Registro	Qué queda registrado en MEMORY.md al terminar.

Ejemplo — flujo de calificación de lead

```
# En AGENTS.md — protocolo de lead entrante:

## Flujo: nuevo lead
# Trigger: webhook recibe datos de un nuevo contacto

$ Pasos:
$ 1. Leer datos del lead: nombre, empresa, tamaño de empresa, mensaje inicial.
$ 2. Clasificar según criterios en MEMORY.md > criterios_calificacion.md:
  - Caliente: empresa +50 empleados Y presupuesto confirmado.
  - Tibio: empresa 10-50 empleados O presupuesto estimado.
  - Frío: no cumple criterios anteriores.
$ 3. Si es caliente: notificar al usuario por WhatsApp inmediatamente con resumen.
  Si es tibio: registrar en MEMORY.md y programar seguimiento en 48h.
  Si es frío: registrar en MEMORY.md, no notificar.
$ 4. Enviar email de bienvenida según la plantilla correspondiente a la clasificación.
$ 5. Registrar en MEMORY.md: [FECHA] Lead [nombre] ([clasificación]) - acción tomada.
```

Flujos que se bifurcan — ramificaciones explícitas

Cuando un flujo puede tomar caminos diferentes según la respuesta del usuario o el resultado de una herramienta, las ramificaciones deben estar documentadas. El modelo no maneja bien la ambigüedad en flujos complejos — necesita que le digas exactamente qué hacer en cada caso.

```
# En AGENTS.md — flujo con ramificaciones:

## Flujo: solicitud de reunión
$ Paso 1: Verificar disponibilidad en Calendar (próximos 5 días hábiles, slots de 45 min).

$ Si hay disponibilidad:
  → Proponer los 2 primeros slots disponibles con formato día/horario.
  → Esperar confirmación antes de crear el evento.

$ Si no hay disponibilidad en 5 días:
  → Notificar al usuario antes de responder al solicitante.
  → Esperar instrucciones: ¿ampliar búsqueda a 10 días? ¿proponer alternativas?

$ Si el solicitante no está en los contactos conocidos:
  → Verificar con el usuario si es un contacto legítimo antes de agendar.
```

VI — Técnicas de prompting que funcionan en OpenClaw

Las técnicas de prompt engineering que funcionan en conversaciones directas con un modelo también funcionan en OpenClaw — pero aplicadas a los archivos del workspace, no a mensajes individuales. Estas son las más documentadas por la comunidad.

Few-shot en AGENTS.md — ejemplos concretos

En lugar de describir el comportamiento esperado, mostrarlo con ejemplos de input y output. El modelo sigue ejemplos concretos con más consistencia que descripciones abstractas.

```
# En AGENTS.md — sección con ejemplos:

## Cómo clasificar prioridades — ejemplos

$ Mensaje: "necesito que revises este contrato para mañana"
$ Clasificación: URGENTE — tarea con deadline explícito
$ Acción: notificar recepción, confirmar que se revisará antes del deadline.

$ Mensaje: "en algún momento me pasás el resumen del mes"
$ Clasificación: NORMAL — sin urgencia
$ Acción: registrar en pendientes, ejecutar en el próximo bloque de trabajo.

$ Mensaje: "cuando puedas buscame algo sobre IA en retail"
$ Clasificación: BAJA — sin deadline, sin urgencia
$ Acción: registrar, ejecutar solo si no hay tareas de mayor prioridad.
```

Chain of thought — razonamiento explícito

Para tareas de análisis o decisiones complejas, instruir al modelo a razonar en voz alta antes de responder produce resultados más consistentes y verificables.

```
# En AGENTS.md — instrucción de razonamiento:

## Análisis de decisiones
$ Antes de recomendar una acción que tenga consecuencias económicas o legales:
$ 1. Enumerar los pros explícitamente.
$ 2. Enumerar los contras explícitamente.
$ 3. Identificar qué información falta para decidir con certeza.
$ 4. Solo entonces dar la recomendación.

# El resultado: el usuario ve el razonamiento y puede corregir
# si el agente tiene una premisa incorrecta, antes de la acción final.
```

Carga de skills bajo demanda — cómo funciona realmente

OpenClaw no inyecta el contenido de los skills en el contexto por defecto — solo inyecta su nombre y descripción. Cuando el agente decide que un skill es relevante, lee el SKILL.md completo en ese momento. Esto es prompt engineering aplicado al sistema: el skill correcto se carga en el momento correcto, sin gastar tokens en skills irrelevantes.

```
# Lo que OpenClaw inyecta por defecto (solo metadata):  
# - nombre del skill  
# - descripcion corta del SKILL.md  
# ~97 chars por skill, independientemente del tamaño del skill  
  
# La instrucción oficial en el sistema prompt de OpenClaw:  
# "Si un solo skill aplica claramente: leer su SKILL.md y seguirlo."  
# "Si varios podrían aplicar: elegir el más específico."  
# "Si ninguno aplica claramente: no leer ningún SKILL.md."  
  
# La implicación para el diseño de skills:  
# La descripcion debe ser precisa y específica sobre cuándo usar el skill.  
# Una descripcion vaga hace que el agente lo cargue en situaciones incorrectas.  
# Una descripcion demasiado amplia compite con otros skills y genera confusión.
```

Testing — cómo saber si las instrucciones funcionan

VII — Testing de prompts y comportamiento

El error más común en prompt engineering es escribir instrucciones y asumir que funcionan. Las instrucciones del agente son código — necesitan ser testeadas. En OpenClaw, el testing se hace de forma deliberada a través de conversaciones de prueba con escenarios específicos.

Qué testear y cómo

Qué testear	Cómo hacerlo	Indicadores de que funciona
Una instrucción nueva en AGENTS.md	Enviar un mensaje que debería activar esa instrucción. Repetirlo con variaciones de redacción.	El agente sigue el comportamiento esperado independientemente de cómo está redactado el mensaje.
Un flujo completo	Simular el escenario completo paso a paso, incluyendo las ramificaciones.	El agente toma las decisiones correctas en cada bifurcación sin necesitar guía adicional.
Un cambio en SOUL.md	Comparar respuestas antes y después del cambio para el mismo tipo de mensaje.	El tono o estilo cambió consistentemente en todos los tipos de mensaje, no solo en los obvios.
La descripción de un skill	Enviar mensajes que deberían activar el skill, y mensajes que no deberían. Verificar cuándo lo carga el agente.	El agente carga el skill cuando corresponde y no lo carga cuando no corresponde.

El ciclo de mejora iterativa

```
# Ciclo de testing en AGENTS.md / SOUL.md:

# 1. Identificar el comportamiento esperado
#   "Cuando llega un email de un cliente, el agente debe..."

# 2. Escribir la instrucción
#   Agregar al archivo correspondiente

# 3. Testear con casos concretos
#   Caso positivo: mensaje que DEBE activar la instrucción
#   Caso negativo: mensaje similar que NO debe activarla
#   Caso extremo: redacción muy diferente del mismo pedido
```

```
# 4. Observar y documentar
# ¿El agente siguió la instrucción?
# ¿Lo hizo en todos los casos o solo en algunos?
# ¿Hubo comportamiento inesperado?

# 5. Refinar
# Si el agente no siguió: la instrucción es ambigua o contradice otra.
# Si solo siguió en algunos casos: agregar ejemplos (few-shot).
# Si produjo comportamiento inesperado: buscar instrucciones que se contradicen.
```

→ Instrucciones que se contradicen — el problema invisible

El error más frecuente en AGENTS.md avanzado es tener instrucciones que se contradicen entre sí. El agente las recibe todas y tiene que elegir — y suele elegir de forma impredecible. Si un comportamiento falla intermitentemente (funciona a veces, no siempre), casi siempre hay una contradicción en el archivo.

Solución: leer AGENTS.md completo buscando instrucciones que puedan entrar en conflicto. Ser específico sobre precedencia: 'La regla X tiene prioridad sobre la regla Y cuando...'

Versionado — tratá los archivos como código

```
# Inicializar git en el workspace:
$ cd ~/.openclaw/workspace
$ git init
$ git add AGENTS.md SOUL.md IDENTITY.md USER.md
$ git commit -m "baseline inicial del agente"

# Antes de cada cambio importante:
$ git add AGENTS.md
$ git commit -m "agregar flujo de calificacion de leads"

# Si algo falla, volver al estado anterior:
$ git checkout HEAD~1 AGENTS.md

# Ver qué cambió entre versiones:
$ git diff HEAD~1 HEAD AGENTS.md
```

→ Siguiendo módulo: Skills Avanzados

Scripts reales en Bash y Python, tipos de skills, seguridad interna y publicación en ClawHub y OpenAgent.lat.